

Guía de Proyecto

Juego por Turnos en JavaFX + Estructuras de Datos

Asignatura	Metodología de la Programación + Estructuras de Datos
Grupo	H12GEXTRA — Marcos Castro · Ventura Pacheco · Marino Rodríguez
Lenguaje	Java 21 · JavaFX · Gson 2.10.1
Entorno	IntelliJ IDEA
Documento	Guía de diseño v1.0 — Sesión inicial de arquitectura

Propósito de este documento

- Sirve como guía de referencia personal durante el desarrollo del proyecto.
- Contiene todas las decisiones de diseño, arquitectura y estructura de clases acordadas.
- Puede entregarse a otra instancia de IA como contexto completo para continuar el desarrollo.
- No es un documento de entrega académica: es una hoja de ruta de trabajo.

1. Descripción del Proyecto

El proyecto consiste en un juego por turnos desarrollado en Java con interfaz gráfica en JavaFX. El mapa del juego es un grafo de habitaciones, donde cada habitación es una matriz de celdas de dimensiones arbitrarias ($N \times M$). El jugador se mueve por las celdas consumiendo puntos de movimiento, combate contra enemigos controlados por IA, gestiona un inventario y puede guardar y cargar la partida en JSON.

Mecánicas principales acordadas

Mecánica	Decisión tomada	Notas
Jugadores	1 jugador vs IA	Sin multijugador
Movimiento	Puntos de movimiento por turno (rango)	BFS sobre <code>Cell[][]</code>
Complejidad	Combate + inventario + varias salas	Sin mecánicas opcionales avanzadas por ahora
Enemigos	3 tipos: Warrior, Archer, Mage	IA distinta por tipo
Items	Weapon, Armor, Potion, SpecialItem	Jerarquía con clase abstracta <code>Item</code>
IA enemiga	Comportamiento distinto según tipo	Método abstracto <code>executeTurn()</code>
Persistencia	JSON con Gson	<code>GameState</code> serializable, grafo plano
Interfaz	JavaFX con MVC	Empieza última, tras tener lógica sólida

Restricciones técnicas del enunciado

- Usar las estructuras de datos propias implementadas en prácticas anteriores (LSE, LDE, Cola, Pila, Grafo, ABB).
- No usar `java.util.*` para estructuras de datos (`ArrayList`, `HashMap`, etc.).
- El grafo de habitaciones debe usar el Grafo genérico `Grafo` ya implementado.
- Tests JUnit para toda la lógica no visual.
- Documentación: UML, memoria PDF, JSON de ejemplo, vídeo demo y diario de uso de IA.

2. Arquitectura General

La arquitectura se organiza en cuatro capas. La regla de oro es que las capas superiores no deben depender de las inferiores en sentido inverso, y JavaFX (capa de presentación) no debe contener lógica de negocio.

Capas del sistema

Capa	Paquete sugerido	Responsabilidad
Estructuras base	MisEstructurasDeDatos	LSE, Cola, Grafo, etc. Ya implementadas. No tocar.
Modelo / Dominio	juego.modelo	Cell, Room, Dungeon, Unit, Player, Enemy, Item, GameState
Lógica del juego	juego.logica	TurnManager, BFSMovimiento, CombateManager, LectorJSON
Presentación JavaFX	juego.vista	Controllers, vistas FXML, GridPane de habitación

Regla de dependencias

Dirección de dependencias (solo hacia abajo)

- Vista → depende de → Lógica
- Lógica → depende de → Modelo
- Modelo → depende de → Estructuras base
- NUNCA: Modelo depende de Vista. NUNCA: Lógica depende de Vista.

Estructura de paquetes recomendada

```
src/  
juego/  
modelo/ → Cell.java, CellType.java, Room.java, Dungeon.java  
modelo/unidad/ → Unit.java, Player.java, Enemy.java, Warrior.java, Archer.java, Mage.java  
modelo/item/ → Item.java, Weapon.java, Armor.java, Potion.java, SpecialItem.java  
logica/ → TurnManager.java, BFSMovimiento.java, LectorJSON.java  
vista/ → MainController.java, RoomView.java, HUDController.java  
MisEstructurasDeDatos/ → (ya existente, no modificar)  
Hoja2b/Grafos/ → (ya existente, usar Grafo directamente)
```

3. Jerarquía de Clases y Relaciones

3.1 Bloque del Mapa

Clase	Tipo	Atributos clave	Relación
CellType	enum	FLOOR, WALL, DOOR, EMPTY	Usado por Cell
Cell	concreta	type: CellType, unit: Unit, item: Item	Compuesta en Room
Room	concreta	id: String, cells: Cell[], rows, cols: int	Compone Cell[], contenida en Dungeon
Dungeon	concreta	grafo: Grafo, currentRoom: NodoGrafo	Envuelve el Grafo existente

Cómo encaja con el Grafo existente

- Dungeon instancia Grafo donde String es la dirección ('norte','sur','este','oeste').
- NodoGrafo es cada habitación. La arista String indica la dirección de la conexión.
- Room debe implementar InterfazDatosNodo (getNombre(), getTipo()) para que buscarNodoPorNombre funcione.
- Dungeon NO extiende Grafo, lo envuelve (composición). Así no expone métodos internos del grafo innecesarios.

3.2 Bloque de Unidades

Clase	Tipo	Atributos clave	Métodos clave
Unit	abstracta	name, hp, maxHp, row, col, roomId: String	takeDamage(int), isAlive(), getPosition()
Player	concreta	movePoints, inventory: LSE, equippedWeapon, equippedArmor	move(r,c), useItem(Item), attack(Unit)
Enemy	abstracta	attackRange, damage: int	executeTurn() — abstracto
Warrior	concreta	hereda Enemy	executeTurn(): mueve hacia jugador y ataca cuerpo a cuerpo
Archer	concreta	hereda Enemy, range ampliado	executeTurn(): dispara desde lejos si tiene línea de visión
Mage	concreta	hereda Enemy, spellDamage	executeTurn(): ataque de área o efecto de estado

Nota importante sobre roomId: Unit NO guarda una referencia directa a Room para evitar referencias circulares al serializar con Gson. En su lugar guarda roomId (String). Al cargar la partida, TurnManager reconstruye la referencia buscando por id en el Dungeon.

3.3 Bloque de Items

Clase	Tipo	Atributos clave	Método use()
Item	abstracta	name, description: String	abstracto — cada subclase lo implementa
Weapon	concreta	damage, range: int	Equipa el arma al jugador, suma daño a ataques
Armor	concreta	defense: int, slot: String	Equipa la armadura, resta daño recibido
Potion	concreta	healAmount: int, effectType: String	Restaura HP o aplica efecto al objetivo
SpecialItem	concreta	effectData: String	Efecto a definir (buff, llave, teletransporte...)

El inventario de Player es LSE (la ListaSimplementeEnlazada ya implementada). La interfaz Usable define use(Unit target) y la implementan todas las subclases de Item.

3.4 Infraestructura del juego

Clase	Responsabilidad	Estructuras que usa
TurnManager	Gestiona el orden de turnos, ejecuta acciones, lleva el log	Cola, LSE
GameState	Snapshot completo de la partida, se serializa a JSON	Contiene Player + Dungeon (plano) + turnNumber
LectorJSON	Carga y guarda GameState desde/hacia fichero JSON	Gson, FileReader, FileWriter
BFSMovimiento	Calcula celdas alcanzables dado un origen y puntos de movimiento	Implementa BFS sobre Cell[][]

4. El BFS de Movimiento — Dos Niveles Distintos

El movimiento en el juego opera en dos niveles completamente distintos. Confundirlos es el error de diseño más común en este tipo de juegos.

Nivel	Opera sobre	Cuándo se usa	Implementación
BFS intra-habitación	Cell[] de la Room actual	Cada vez que el jugador quiere moverse dentro de la sala. Calcula qué celdas son alcanzables con los movePoints disponibles.	BFSMovimiento.java — nuevo, a implementar
BFS inter-habitación	Grafo del Dungeon	Para encontrar el camino entre salas (IA persiguiendo jugador, navegación automática).	Ya implementado en Grafo.caminoMinimo()

Pseudocódigo BFS intra-habitación

```
getCellsInRange(startRow, startCol, movePoints, room):  
cola = Cola vacía  
visitadas = Set vacío  
cola.enqueue({row: startRow, col: startCol, ptsRestantes: movePoints})  
alcanzables = LSE vacía  
mientras cola no vacía:  
  actual = cola.dequeue()  
  si actual ya visitada → continuar  
  marcar actual como visitada  
  alcanzables.addEnd(actual)  
  para cada vecino NSEO de actual:  
    si vecino es FLOOR y ptsRestantes > 0:  
      cola.enqueue({vecino, ptsRestantes - 1})  
return alcanzables
```

El coste de moverse entre celdas puede ser 1 por defecto. Si quieres terreno con coste variable (agua = 2 puntos, barro = 3...) simplemente cambia el '1' por 'cell.moveCost'. Diseña la clase Cell con ese atributo desde el principio aunque al inicio todos sean 1.

5. Roadmap de Desarrollo por Fases

El orden de desarrollo es crítico. No empieces por JavaFX. No empieces por el JSON. La regla es: lógica sólida primero, UI después. Cada fase termina con tests antes de pasar a la siguiente.

Fase 0 — Diseño previo

- UML de clases (Capa 1: herencia, Capa 2: atributos) — COMPLETADO en sesión inicial.
- Bocetos de UI en papel (no código).
- Documento de reglas del juego en prosa.
- Checklist de validación contra el enunciado.

Fase 1 — Núcleo de datos (EMPEZAR AQUÍ)

- CellType (enum): FLOOR, WALL, DOOR, EMPTY.
- Cell: type, unit, item, moveCost.
- Room: Cell^{[][]}, id, nombre, getCell(r,c), setUnit(r,c,u).
- Dungeon: envuelve Grafo, addRoom(), connect(r1,r2,dir).
- Unit (abstracta), Player, Enemy (abstracta), Warrior, Archer, Mage.
- Item (abstracta), Weapon, Armor, Potion, SpecialItem.
- TurnManager: Cola, nextTurn(), addLog().
- → Tests JUnit para todas estas clases antes de continuar.

Fase 2 — Lógica del juego

- BFSMovimiento: getCellsInRange() sobre Cell^{[][]}.
- Sistema de movimiento del jugador: consumir movePoints, cambiar de sala si celda es DOOR.
- Sistema de combate: Player.attack(Enemy), takeDamage(), muerte de unidad.
- executeTurn() para cada tipo de Enemy (Warrior, Archer, Mage).
- Sistema de inventario: añadir, usar, equipar item.
- → Tests JUnit para BFS, combate y turnos.

Fase 3 — Persistencia JSON

- GameState: estructura plana serializable (ver sección de advertencias).
- LectorJSON.guardar(GameState, ruta) y LectorJSON.cargar(ruta).
- Manejo de excepciones de I/O (IOException, JsonSyntaxException).
- JSON de ejemplo para entregar con el proyecto.
- → Tests JUnit para guardar/cargar y casos de error.

Fase 4 — Interfaz JavaFX

- Vista de habitación: GridPane dinámico que dibuja Cell[][].
- Resaltado de celdas alcanzables (BFS result) al seleccionar jugador.
- Panel lateral: stats del jugador, inventario, turno actual.
- Log de acciones: ScrollPane con historial de TurnManager.log.
- Pantalla de fin de partida (victoria/derrota).
- Menú principal: nueva partida, cargar partida.
- → Pruebas manuales. No hay tests JUnit para la vista.

6. Checklist de Requisitos y Oportunidades de Mejora

6.1 Requisitos del enunciado

Requisito	Estado	Cómo se cumple
Cola para gestionar turnos	✓ Diseñado	TurnManager usa Cola existente
Grafo de habitaciones	✓ Diseñado	Dungeon envuelve Grafo
Matriz NxM por habitación	✓ Diseñado	Room tiene Cell[][]
BFS para movimiento	✓ Diseñado	BFSMovimiento sobre Cell[][]
Inventario con lista propia	✓ Diseñado	Player.inventory es LSE
Herencia y polimorfismo	✓ Diseñado	Unit→Player/Enemy, Item→subclases
Serialización JSON	✓ Diseñado	GameState + Gson
Log de operaciones	✓ Diseñado	TurnManager.log es LSE
Tests JUnit lógica	■ Pendiente	Una vez implementadas las clases
Manejo de excepciones propio	■ Pendiente	InvalidMoveException, etc.
UML entregable	✓ Borrador	Capa 1 y Capa 2 generadas en sesión inicial
Memoria PDF académica	■ Pendiente	Fase final del proyecto
Vídeo demo	■ Pendiente	Grabación de partida funcional
Diario de uso de IA	■ En curso	Registrar cada sesión con IA

6.2 Oportunidades de mejora y ampliación

Estos puntos van más allá del mínimo exigido y pueden marcar la diferencia en la nota. Se listan por orden de impacto vs esfuerzo.

Mejora	Impacto	Esfuerzo	Descripción
Coste de movimiento variable	Alto	Bajo	Añadir moveCost a Cell desde el principio. Permite terreno con agua, barro, etc.
Árbol de habilidades (ABB)	Alto	Medio	El jugador sube de nivel y desbloquea habilidades almacenadas en un ABB ordenado por coste.
Sistema de niebla de guerra	Alto	Medio	Las celdas no visitadas aparecen ocultas. Añadir Cell.visited: boolean.
Generación procedural del mapa	Alto	Alto	Algoritmo que genera el grafo de salas y sus matrices aleatoriamente al iniciar.
IA con BFS de persecución	Medio	Medio	Warrior usa BFS intra-habitación para encontrar el camino más corto al jugador.

Mejora	Impacto	Esfuerzo	Descripción
Múltiples pisos / niveles	Medio	Medio	El Dungeon tiene varios subgrafos conectados por escaleras. Amplía el grafo con nodo especial.
Efectos de estado	Medio	Bajo	Envenenado, paralizado, etc. Añadir ListaSimplementeEnlazada a Unit.
Animaciones JavaFX	Bajo	Alto	Transiciones entre celdas y salas. Añadir valor estético pero no aporta nota técnica.
Estadísticas de partida	Bajo	Bajo	Al terminar, mostrar turnos jugados, enemigos eliminados, daño total. Datos ya en el log.

7. Advertencias y Decisiones Críticas

Esta sección recoge los puntos donde un error de diseño temprano obliga a reescribir grandes cantidades de código. Léelos antes de escribir las primeras clases.

ADVERTENCIA 1 — Serialización del Grafo con Gson

- Problema: Gson no puede serializar Grafo directamente porque tiene referencias circulares (NodoGrafo apunta a Arista, Arista apunta a NodoGrafo).
- Solución adoptada: GameState NO guarda el objeto Grafo. En su lugar guarda dos listas planas:
- → List: id, nombre, filas, cols, y la matriz de celdas serializada.
- → List: idOrigen, idDestino, direccion (String).
- Al cargar, LectorJSON reconstruye el Dungeon desde esas listas planas.
- NO implementes una alternativa (JsonSerializer custom) sin antes consultar. El serializer custom de Gson es complejo y propenso a errores con generics de Java.

ADVERTENCIA 2 — Referencias circulares Unit ↔ Room

- Problema: Si Unit tiene Room room y Room tiene Cell[][] con Unit unit, Gson entra en bucle infinito.
- Solución adoptada: Unit guarda String roomId, NO la referencia al objeto Room.
- Al cargar la partida: TurnManager.reconstruirReferencias() recorre todas las unidades, busca su Room en el Dungeon por id y reestablece la referencia en memoria.
- Aplicar esto desde el primer día. Cambiarlo después implica refactorizar Player, todos los Enemy y TurnManager al mismo tiempo.

ADVERTENCIA 3 — No empezar por JavaFX

- El error más frecuente en proyectos similares es montar la UI antes de que la lógica esté sólida.
- Consecuencia: se descubren fallos de arquitectura cuando ya hay código de vista mezclado con lógica.
- Regla: la Fase 4 (JavaFX) no empieza hasta que las Fases 1, 2 y 3 pasen todos sus tests.
- La UI debe ser un observador pasivo del modelo, no contener ningún cálculo de juego.

ADVERTENCIA 4 — Room debe implementar InterfazDatosNodo

- El método buscarNodoPorNombre() del Grafo existente llama a ((InterfazDatosNodo)datos).getNombre().
- Si Room no implementa InterfazDatosNodo, ese método lanza ClassCastException en runtime.
- Solución: Room implements InterfazDatosNodo. getNombre() devuelve el id de la sala, getTipo() devuelve 'room'.
- Alternativa: sobrescribir buscarNodoPorNombre en Dungeon con lógica propia. Más trabajo, menos reuso.

ADVERTENCIA 5 — BFS y el turno del jugador deben estar desacoplados

- El BFS de movimiento calcula qué celdas son ALCANZABLES, pero no mueve al jugador.
- El movimiento real se ejecuta solo cuando el jugador hace click en una celda alcanzable.
- Si mezclas el cálculo con el movimiento, no podrás mostrar el resaltado de celdas en JavaFX sin ejecutar el movimiento accidentalmente.
- `BFSMovimiento.getCellsInRange()` devuelve una lista. `Player.move(r,c)` ejecuta el movimiento. Son dos operaciones separadas.

8. Instrucciones para la IA en Conversaciones Futuras

Esta sección está dirigida a la instancia de IA que reciba este documento como contexto. Define cómo debe comportarse para ser útil en el desarrollo de este proyecto.

Contexto del proyecto

- El proyecto es un juego por turnos en Java + JavaFX. La arquitectura está diseñada.
- El alumno tiene código base de prácticas anteriores: Grafo, LSE, Cola, Pila, ABB — todo funcionando.
- NO hay que reimplementar estructuras de datos. Hay que usar las existentes.
- El alumno es estudiante universitario, nivel Java intermedio-avanzado.

Cómo ayudar en conversaciones de desarrollo

- Cuando el alumno pide implementar una clase: escribe el código completo, no fragmentos.
- Siempre que generes código, verifica que usa LSE/Cola/Grafo propias, no `java.util.*`.
- Si detectas un error de diseño que contradice este documento, dilo directamente antes de continuar.
- No repitas lo que ya está decidido. Asume que el alumno conoce la arquitectura de este PDF.
- Cuando haya varias opciones de implementación, da tu recomendación directa con justificación breve.
- Si el alumno propone algo que rompe una advertencia de la Sección 7, señálalo con claridad.

Orden de implementación recomendado dentro de Fase 1

- 1. `CellType.java` (enum, 5 min)
- 2. `Cell.java` (atributos + getters, 15 min)
- 3. `Room.java` (constructor con filas/cols, `getCell`, `setUnit`, 30 min)
- 4. `Room` implements `InterfazDatosNodo` (`getNombre`, `getTipo`)
- 5. `Dungeon.java` (envuelve Grafo, `addRoom`, `connect`, 45 min)
- 6. `Unit.java` abstracta, `Player.java` concreta
- 7. `Enemy.java` abstracta, `Warrior/Archer/Mage` concretas
- 8. `Item.java` abstracta, `Weapon/Armor/Potion/SpecialItem`
- 9. Tests JUnit para todo lo anterior
- 10. `TurnManager.java` (Cola, log, `nextTurn`)

Cosas que NO debe hacer la IA

- No sugerir usar `ArrayList`, `HashMap` o cualquier estructura de `java.util` para lógica del juego.

- No proponer empezar por JavaFX antes de que la lógica esté completa y testeada.
- No implementar un JsonSerializer custom de Gson sin que el alumno lo solicite explícitamente.
- No añadir Unit.room (referencia directa) en lugar de Unit.roomId (String).
- No mezclar lógica de juego en los controllers de JavaFX.
- No dar la razón por defecto si el alumno propone algo que rompe las advertencias de la Sección 7.